

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(UGC-Autonomous Institution)

Ghanpur(V), Ghatkesar(M), Medchal(D), Hyderabad-500088

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (AI&ML)

MAJOR PROJECT DOCUMENTATION

MCP-BASED PERSONAL PRODUCTIVITY AGENT

An Intelligent AI Assistant Powered by Model Context Protocol

Under the Guidance of

HoD, Dr. PULIME SATYANARAYANA,
Associate Professor of CSE(AIML)

Presented by

Name	Roll Number
Krishna Keerthana Potharaju	23RA1A6641
Soma Ganesh	23RA1A6627
Nikhil Kokkula	23RA1A6662
Malishka Reddy Gone	23RA1A6647

1. SYSTEM ARCHITECTURE

The MCP-Based Personal Productivity Agent follows a layered architecture in which each layer has a clearly defined responsibility. The user provides a natural-language productivity request, and the LangGraph Agent controls the reasoning workflow. The LLMClient supports intent understanding and response generation, while the MCP Client provides standardized tool discovery and invocation. The Unified MCP Server acts as the single gateway for 17 productivity tools covering Tasks, Calendar, and Notes. These tools remain thin and delegate business logic to the corresponding service layer. The service layer performs validation, business rules, conflict handling, search, and persistence through SQLAlchemy and SQLite. This separation keeps reasoning, protocol communication, domain logic, and storage independently testable and extensible.

Layer	Responsibility
Presentation / User Interaction	Accepts natural-language productivity requests and displays the final response.
Agent / Reasoning Layer	Understands intent, selects tools, prepares arguments, validates execution, and controls the workflow using LangGraph.
LLM Integration Layer	Provides the LLMClient abstraction for structured request interpretation and final response generation.
MCP Client	Connects through stdio, discovers capabilities dynamically, invokes tools, and manages the MCP session lifecycle.
Unified MCP Server	Provides one standardized interface exposing exactly 17 productivity tools.
Domain MCP Tools	Translate MCP requests into Task, Calendar, and Notes service operations without duplicating business logic.
Service / Validation Layer	Owns input validation, business rules, date handling, conflict detection, search, persistence, and controlled errors.
Database Layer	Uses SQLAlchemy ORM with SQLite to store and retrieve tasks, calendar events, and notes.

Table 1: Layered architecture and responsibilities

Architecture interpretation: The key design principle is separation of concerns. The agent determines what the user wants, the MCP layer provides standardized capability access, the service layer determines how the business operation is performed, and the database layer persists the result. The Unified MCP Server does not directly implement domain business logic or database queries.

2. MODULE ARCHITECTURE



Figure 1: Major module flow of the MCP-based productivity system

The module structure follows the implemented layered flow of the system. Natural-language input enters through the user interaction layer and is processed by the AI Agent and LLM integration. The MCP Client provides dynamic capability discovery and invocation through the Unified MCP Server. The server exposes the Task, Calendar, and Notes productivity tools, which delegate domain operations to the service and validation layer. Finally, SQLAlchemy and SQLite provide persistent storage. This separation keeps reasoning, protocol communication, business logic, and data storage independent and reusable.

2.1 MODULE DESCRIPTION

The following module descriptions represent the major functional parts of the implemented system.

1) User Interaction

Accepts natural-language productivity requests and displays the final response. The user can request task, calendar, or notes operations without knowing the underlying MCP tool names.

2) AI Agent & Reasoning

Understands user intent and determines the required action using LangGraph. It coordinates tool discovery, selection, argument preparation, validation, invocation, result handling, and response generation.

3) MCP Client

Connects with MCP servers, initializes sessions, discovers available tools dynamically, invokes the selected tool with structured arguments, and manages connection lifecycle and failures.

4) Unified MCP Server

Provides a single standardized interface for all productivity tools. It exposes exactly 17 tools: 6 Task tools, 5 Calendar tools, and 6 Notes tools.

5) Productivity Tools

Handles Tasks, Calendar, and Notes operations according to the user's request. The tools provide CRUD capabilities, task completion, and note search while delegating business logic to services.

6) Service & Validation

Performs input validation, business rules, date/time handling, conflict detection, search, persistence, and controlled error handling. It remains the source of truth for domain behaviour.

7) Database

Stores and retrieves tasks, calendar events, and notes using SQLAlchemy ORM with SQLite persistent storage.

3. UML DIAGRAMS

UML (Unified Modeling Language) diagrams describe the structure and behaviour of the system visually. Six diagrams are used here: the use case diagram for functionality, the class diagram for static structure, the sequence diagram for message order, the activity diagram for workflow, the component diagram for architecture, and the ER diagram for the persistent data model.

3.1 Use Case Diagram

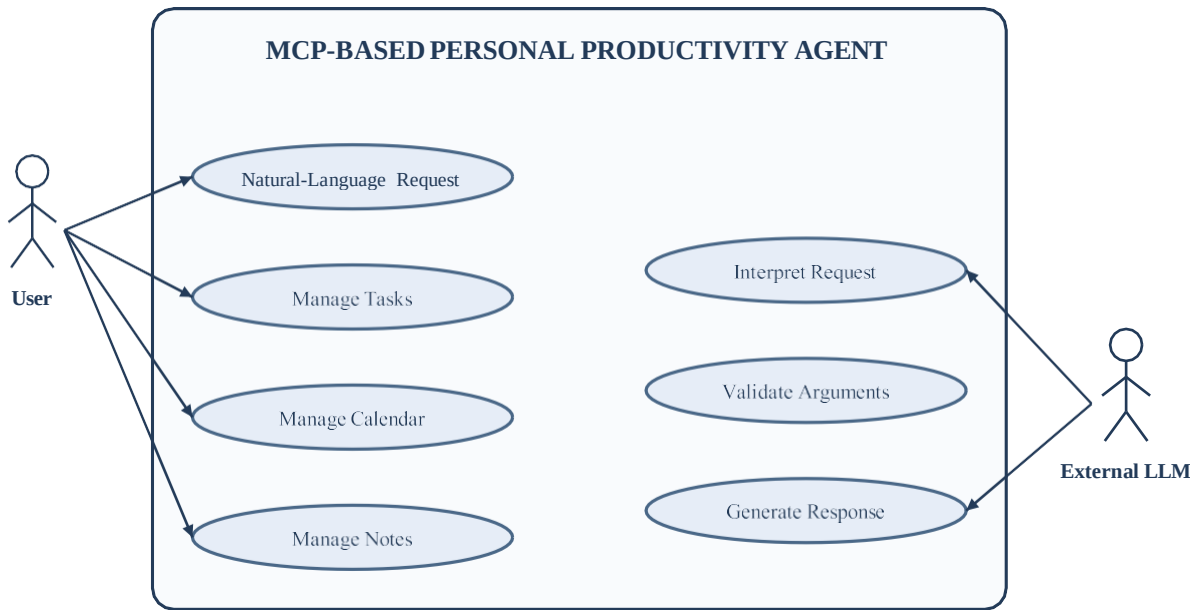


Figure 2: Use case diagram of the MCP-based personal productivity agent

The use case diagram shows the functional requirements from the point of view of the user and the external language-model dependency. The user initiates natural-language productivity requests and can work with Tasks, Calendar, and Notes. The agent-side use cases cover request interpretation, validation, and response generation, while the external LLM provider supports language understanding and response generation. Internal components such as the MCP Client, services, and database are implementation components rather than UML actors. This keeps the system boundary clear and focuses the diagram on user-visible functionality.

3.2 Class Diagram

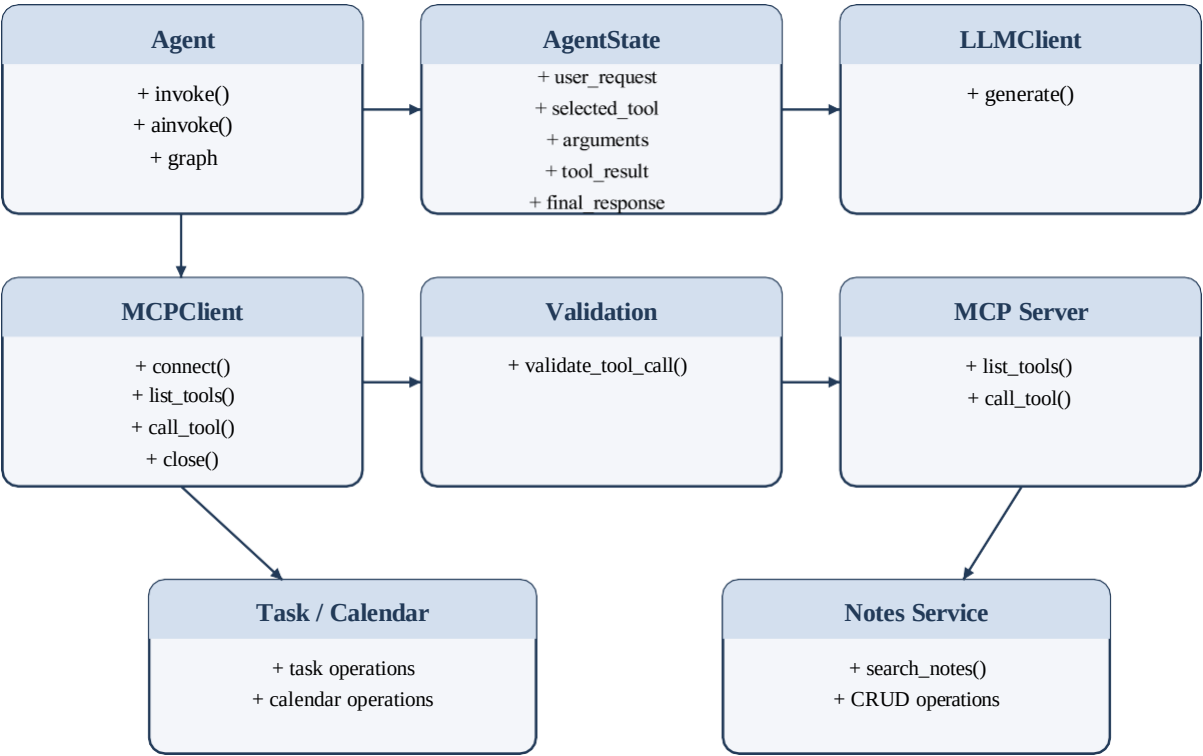


Figure 3: Class diagram of the agent, MCP and service structure.

The class diagram presents the static structure of the core software abstractions. Agent controls the LangGraph workflow, while AgentState carries the information needed between workflow steps. LLMClient isolates language-model communication, MCPClient handles connection and tool invocation, and Validation provides the pre-execution validation boundary. The MCP Server exposes capabilities and delegates domain work to Task, Calendar, and Notes services. The database is accessed through the service layer using SQLAlchemy and SQLite, keeping persistence separate from agent logic.

3.3 Sequence Diagram

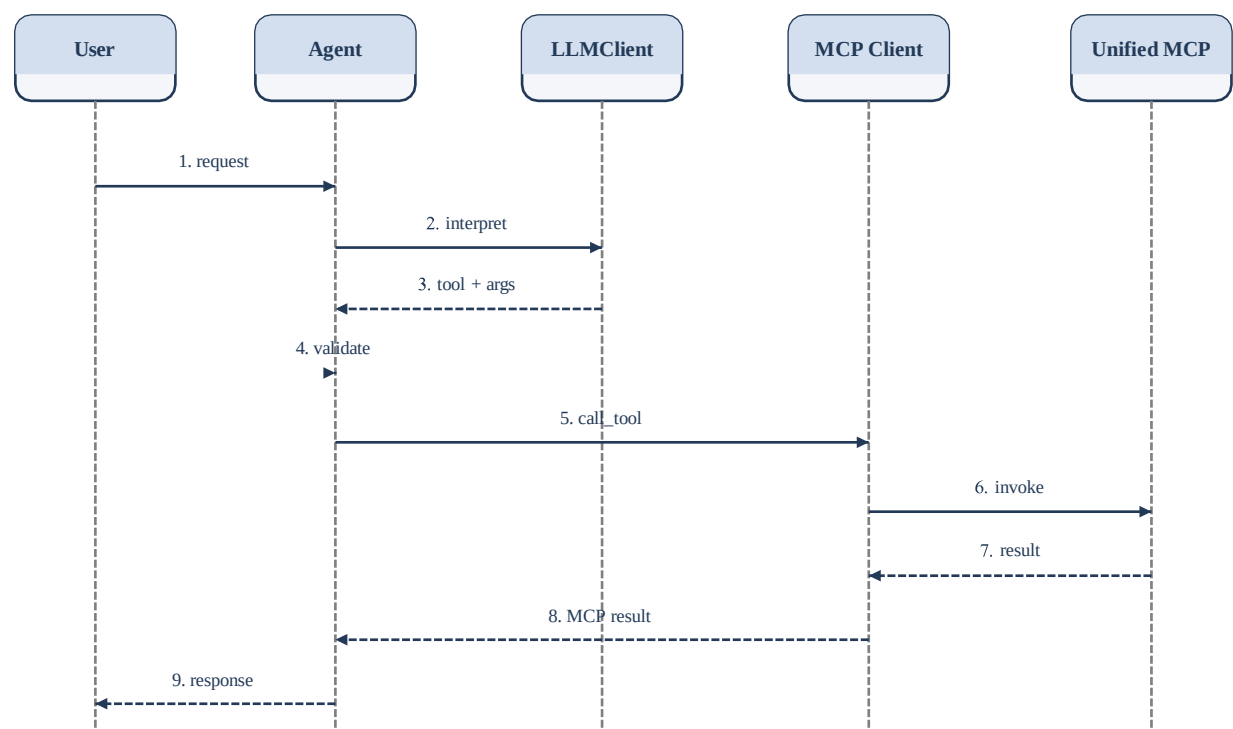


Figure 4: Sequence diagram of one natural-language productivity request

The sequence diagram shows the order of messages for a single productivity request. The user sends a natural-language request to the agent, which uses the LLMClient to interpret the request and determine the tool and arguments. The agent validates those arguments before the MCP Client invokes the selected tool on the Unified MCP Server. The server-side operation returns a structured result through the MCP connection. The agent then converts that result into the final response shown to the user.

3.4 Activity Diagram

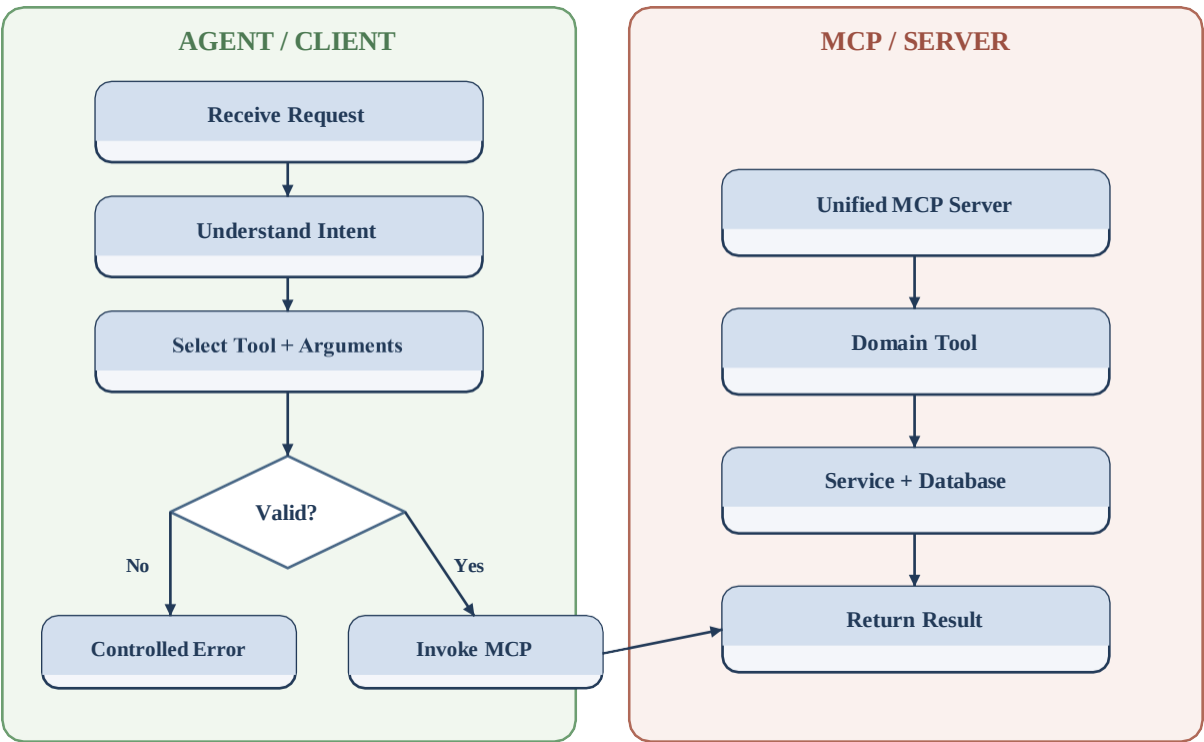


Figure 5: Activity diagram of the end-to-end natural-language workflow

The activity diagram represents the complete control flow from request to response. The agent receives the request, understands the intent, selects the appropriate tool, and prepares its arguments. A decision node checks whether the arguments are valid. Invalid input produces a controlled error, while valid input continues to MCP invocation. The Unified MCP Server routes the call to the domain tool and service, and the resulting data returns to the agent for final response generation.

3.5 Component Diagram

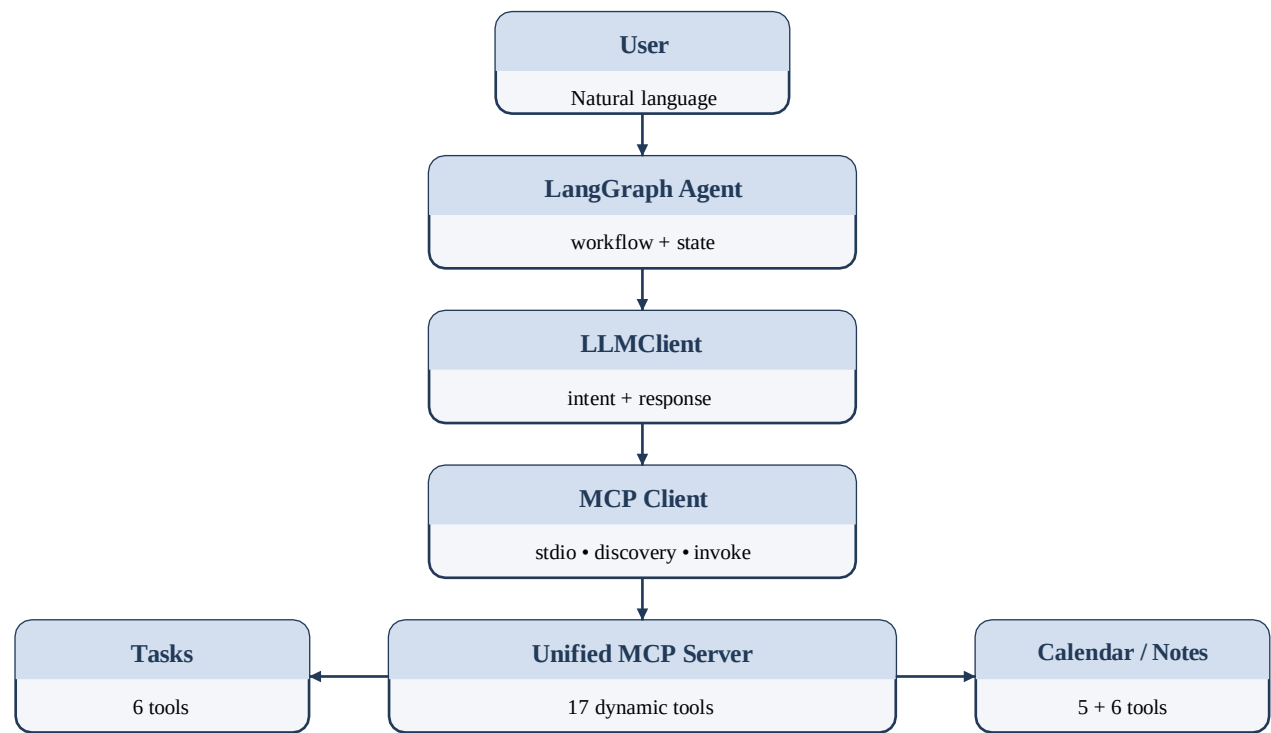


Figure 6: Component diagram of the layered MCP agent architecture

The component diagram presents the runtime architecture and dependency direction between the major components. The LangGraph Agent and LLMClient handle reasoning and language interaction, while the MCP Client handles standardized capability access. The Unified MCP Server exposes 17 tools and routes them to the Task, Calendar, and Notes domains. The service layer contains business rules and validation, while SQLAlchemy and SQLite provide persistence. This separation prevents the agent from directly accessing the database and keeps the MCP server thin.

3.6 ER Diagram

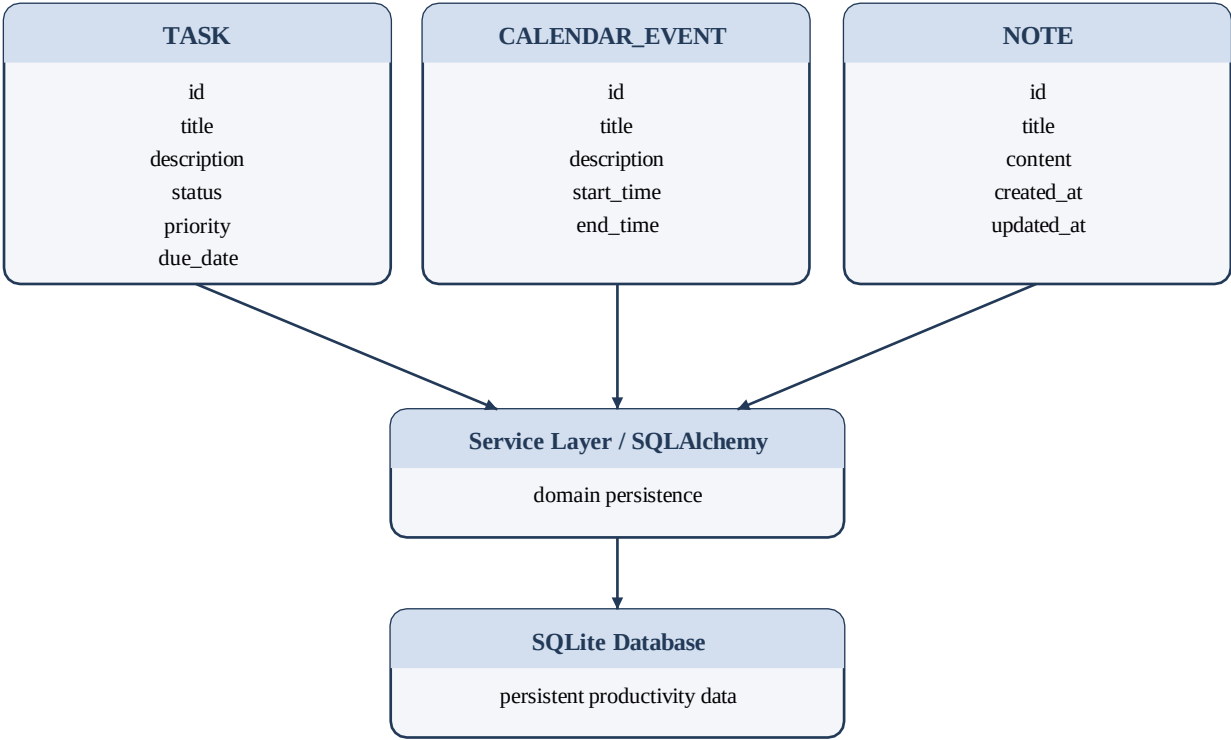


Figure 7: ER-style data model of the productivity domains

The ER-style diagram represents the three core persistent productivity entities. Task stores information such as title, status, priority, and due date. Calendar Event stores scheduling information through start and end times, while Note stores textual content and timestamps. Each domain is handled by its corresponding service and exposed through MCP tools. SQLAlchemy maps these application entities to SQLite persistence, keeping the data layer separate from the agent and protocol layers.

4. DESIGN SUMMARY

The six UML views collectively document the project from complementary perspectives. The use case diagram explains what the user can accomplish, the class diagram shows the main software abstractions, and the sequence diagram explains message order during execution. The activity diagram presents the decision and validation workflow, the component diagram shows the runtime architecture, and the ER diagram presents the persistent Task, Calendar, and Notes data model. Together, these views describe how the LangGraph agent, LLMClient, MCP Client, Unified MCP Server, 17 productivity tools, service layer, and SQLite persistence cooperate as one modular productivity system.